

ello

Kurumsal Mesajlaşma Uygulaması

Kaynak Kod, Mimari, Uygulama Özellikleri
ve Teknik Karar Raporu

İncelenen sürüm

Branch: codex/post-handoff-demo-fixes

Commit: 31bfd72

Tarih: 27 Temmuz 2026

Kapsam: React frontend, NestJS backend, PostgreSQL/Prisma, Socket.IO, WebRTC, Spring Boot Java webhook,
Docker/Nginx ve test altyapısı.

ello

Doküman Hakkında

Bu rapor, elIO kod deposunun çalışan durumunu teknik ve ürün bakış açısıyla bir araya getirir. Amaç yalnızca kullanılan kütüphaneleri saymak değil; uygulamanın hangi problemi çözdüğünü, modüllerin nasıl haberleştiğini, özelliklerin hangi iş kurallarıyla çalıştığını, neden belirli teknolojilerin seçildiğini ve geliştirme boyunca hangi sorunların neden çözüldüğünü açıklamaktır.

Doğruluk ilkesi

Raporda tamamlanmış ve test edilmiş davranışlar ile yalnızca arayüz şablonunda bulunan veya kalıcı backend desteği olmayan seçenekler ayrı gösterilir. Bu nedenle doküman, sunum metni olmasının yanında teknik durum tespiti olarak da kullanılabilir.

Alan	İncelenen kaynak
Kod tabanı	Backend, frontend, Java webhook, Prisma şeması ve 13 migration
Davranış	REST controller/service kuralları, Socket.IO gateway ve WebRTC istemci akışı
Operasyon	Docker Compose, üç Dockerfile, Nginx proxy ve environment doğrulaması
Kalite	Jest, Supertest, Playwright, JUnit ve full-stack test betikleri
Geçmiş	Git commit geçmişi, daily log, handoff ve release dokümanları

İçindekiler

1. Yönetici Özeti	5
1.1 Ürünün temel değeri	5
1.2 Güncel doğrulama durumu	5
2. Ürün Kapsamı ve Kullanıcı Roller	5
2.1 Hazır demo hesapları	6
3. Sistem Mimarisi	6
3.1 İstek yolu	7
3.2 Neden same-origin proxy?	7
3.3 Geliştirme ve production-benzeri mod	7
4. Teknoloji Seçimleri ve Gerekçeleri	7
4.1 Bilinçli mimari tercihlerin sonucu	8
5. Depo ve Kod Organizasyonu	8
5.1 Backend modül sınırları	9
6. Backend Derin İnceleme	9
6.1 Global request pipeline	9
6.2 Kimlik doğrulama	9
6.3 Conversation kuralları	9
6.4 Mesaj güvenilirliği	10
6.5 Attachment doğrulaması	10
7. Veritabanı Tasarımı	10
7.1 Enum sözlüğü	11

7.2 İlişki ve silme kararları	11
7.3 İndeks stratejisi	11
8. REST API Referansı	11
8.1 Standart response ve hata biçimi	13
9. Socket.IO ve Gerçek Zamanlı Akış	13
9.1 İstemciden sunucuya event'ler	13
9.2 Sunucudan istemciye event'ler	14
9.3 Presence ve reconnect	14
10. Frontend Mimarisi	14
10.1 Dashboard navigasyonu	14
11. Uygulamanın Özellikleri	15
11.1 Authentication ve oturum	15
11.2 Direct mesajlaşma ve Contacts	16
11.3 Grup ve channel yönetimi	16
11.4 Mesaj özellikleri	16
11.5 Conversation listesi	16
11.6 Birebir sesli arama	16
11.7 Support ticket yönetimi	17
11.8 Profil, tema ve yardım	17
11.9 Responsive ve erişilebilirlik	17
12. Java Webhook Servisi	17
12.1 Akış	17
12.2 Health ve runtime	18
13. Bot Otomasyonu	18
13.1 Ticket'tan gruba örnek	18
14. Güvenlik ve Dayanıklılık	18
14.1 Bilinen güvenlik borcu	19
15. Docker, Ağ ve Operasyon	19
15.1 Secret ve environment yönetimi	19
15.2 Backup, restore ve recovery	20
15.3 Çalıştırma	20
16. Test Stratejisi ve Kalite Güvencesi	20
16.1 Playwright'ta doğrulanan kullanıcı senaryoları	20
16.2 Son doğrulama özeti	21
17. Geliştirme Sürecinde Neler Yapıldı ve Neden?	21
17.1 Kapatılan dört kritik kullanıcı bug'ı	22
17.2 Son demo işlev tamamlama paketi	22

18. Tamamlanmamış veya Sınırlı Alanlar	22
19. Önerilen Yol Haritası	23
20. Demo ve Test Runbook	23
20.1 Otomatik kontrol	23
20.2 Manuel iki oturum kontrolü	23
20.3 Sunum sonrası	23
Ek A. Veri Akışı Örnekleri	24
A.1 Direct mesaj	24
A.2 Attachment	24
A.3 Audio call	24
Ek B. Kaynak Kod Referans Haritası	24
Ek C. Komut Referansı	25
Ek D. Sonuç	25

1. Yönetici Özeti

ello, kurum içi mesajlaşma senaryosunu hedefleyen; doğrudan ve grup sohbetlerini, yönetici rolleri ile destek taleplerini, dış sistem otomasyonunu ve birebir sesli aramayı tek bir uygulamada birleştiren tam yığın bir demo üründür.

Sistem, tarayıcıdaki React arayüzünün Nginx üzerinden NestJS API ve Socket.IO katmanına bağlanmasıyla çalışır. İş verileri PostgreSQL'de Prisma ile kalıcıdır. Dış ticket olayları, Java/Spring Boot adaptörü üzerinden bot kontrollü gruplara dönüştürülür. Ses medyası WebRTC ile kullanıcılar arasında akar; backend yalnızca yetkilendirme, sinyal iletimi ve çağrı durumunu yönetir.

Gösterge	Mevcut durum
Backend kaynak kodu	117 TypeScript dosyası, yaklaşık 9.750 satır
Frontend kaynak kodu	198 TS/TSX dosyası, yaklaşık 19.981 satır
Stil katmanı	46 SCSS dosyası, yaklaşık 28.010 satır
Java servisi	10 ana Java dosyası, yaklaşık 299 satır
Veritabanı	11 model, 13 migration, son audit: 34 indeks ve 13 foreign key
Test kapsamı	Backend unit/e2e, frontend unit/Playwright, Java JUnit ve full-stack smoke
Çalışan servisler	PostgreSQL, NestJS, Java webhook ve Nginx/React frontend

1.1 Ürünün temel değeri

- Tek arayüz: sohbet, kişi, çağrı, kayıtlı mesaj, destek ve ayarlar aynı dashboard içinde sunulur.
- Gerçek zamanlı davranış: mesaj, düzenleme, silme, typing, read receipt, presence ve grup değişiklikleri yenileme gerektirmeden taşınır.
- Kurumsal grup yönetimi: owner/manager/member rolleri, özel manager chat, mesaj ve ayrılma politikaları vardır.
- Dış sistem entegrasyonu: ticket webhook'u, idempotent bot grubu ve başlangıç mesajı oluşturabilir.
- Kalıcı ve test edilebilir mimari: PostgreSQL migration'ları, audit araçları ve tek komut full-stack testi vardır.

1.2 Güncel doğrulama durumu

27 Temmuz 2026 tarihli son kontrolde full-stack test başarıyla tamamlanmıştır. Kontrol; frontend ve same-origin proxy, backend health, Swagger/OpenAPI, Java health/readiness, admin rolü, kullanıcı yetki sınırı, direct ve group Socket.IO teslimi, retry idempotency, attachment upload/download, ticket koordinasyonu ve webhook kimlik doğrulamasını kapsamıştır.

Demo durumu

Manuel iki oturum testi de geçmiştir: admin ve kullanıcı hesaplarıyla grup mesajları yenileme gerektirmeden iki tarafa ulaşmıştır. Swagger Compose ortamında varsayılan olarak açıktır; demo kullanıcıları bootstrap servisiyle korunur.

2. Ürün Kapsamı ve Kullanıcı Roller

Uygulamada iki global rol ve grup bazında üç katılımcı rolü bulunur. Global rol, uygulama çapındaki yetkiyi; katılımcı rolü ise belirli bir grup içindeki yetkiyi belirler.

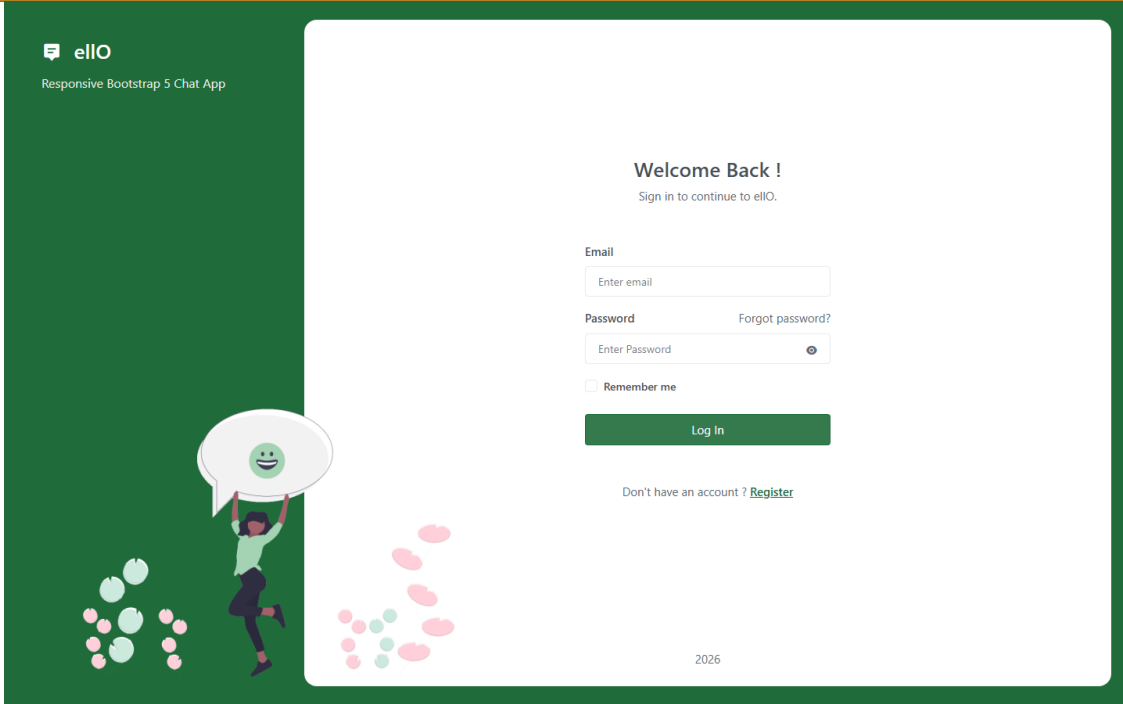
Rol	Kapsam	Başlıca yetkiler
Global admin	Uygulama	Grup oluşturma, tüm destek taleplerini görme, ticket atama/çözme, runtime metrics
Global user	Uygulama	Direct sohbet, davet, kendi ticket'ını oluşturma/görme, katıldığı grupları kullanma
Group owner	Tek grup	Grup ayarları, sahiplik devri, manager/member yönetimi; sahipliği devretmeden ayıramaz
Group manager	Tek grup	İzin verilen grup yönetimi ve özel manager chat erişimi
Group member	Tek grup	Politikalara göre mesaj gönderme ve gruptan ayrılma

2.1 Hazır demo hesapları

Automation ID	E-posta	Rol	Şifre
1	emiradmin@ello.com	admin	123456
2	emiruser@ello.com	user	123456
3	asliadmin@ello.com	admin	123456
4	asliuser@ello.com	user	123456
5	gulsimaadmin@ello.com	admin	123456
6	gulsimauser@ello.com	user	123456

Demo hesabı uyarısı

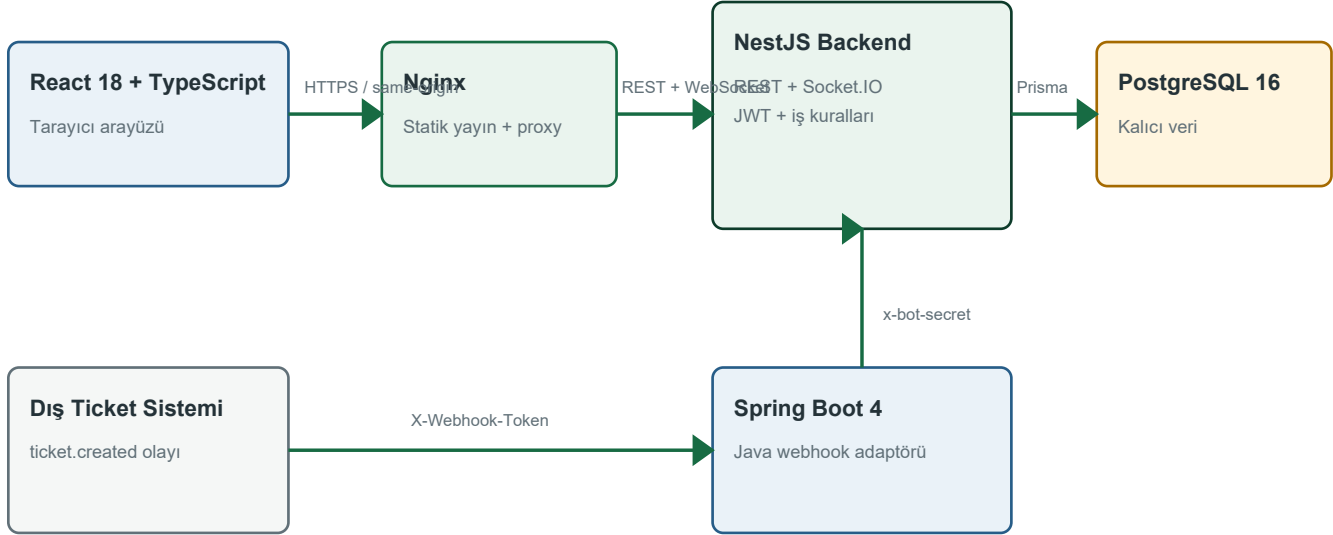
Bu hesaplar yerel demo ve sunum kolaylığı içindir. `123456` production parolası değildir. Geçici bir internet tüneli açılırsa bağlantı yalnızca hedef testçilere verilmeli ve test sonrası tünel durdurulmalıdır.



Şekil 1 - ello giriş ekranı.

3. Sistem Mimarisi

Mimari, kullanıcı arayüzü, iş mantığı, gerçek zamanlı iletişim, kalıcı veri ve dış entegrasyon sorumluluklarını ayrı katmanlara böler.



Ses medyası WebRTC ile kullanıcılar arasında akar; backend yalnızca sinyal ve çağrı durumunu yönetir.

Şekil 2 - ello bileşenleri ve temel trafik yönleri.

3.1 İstek yolu

- 1 Tarayıcı React uygulamasını `:5173` üzerindeki Nginx container'ından alır.
- 2 Frontend REST çağrılarını `/api`, Socket.IO bağlantısını `/chat` taban adresiyle aynı origin üzerinden başlatır.
- 3 Nginx API ve WebSocket upgrade trafiğini Compose ağı içindeki NestJS backend'e iletir.
- 4 NestJS validation, authentication, role/membership kontrolü ve iş kurallarını uygular.
- 5 Prisma kullanılan modüllerde veriyi PostgreSQL'e yazar; realtime event service değişikliği Socket.IO katmanına taşır.
- 6 İstemciler yeni state'i Redux/Saga ve component state üzerinden arayüze yansıtır.

3.2 Neden same-origin proxy?

Frontend build'ine bilgisayarın değişken LAN IP adresini gömmek yerine `/api` ve `/chat` görelî adresleri kullanılır. Böylece masaüstü, laptop veya aynı ağdaki ikinci cihaz yalnızca `http://HOST\_IP:5173` adresini açar. CORS karmaşası azalır, Socket.IO upgrade akışı tek giriş noktasından geçer ve backend portunu yerel makine dışına açmak gerekmez.

3.3 Geliştirme ve production-benzeri mod

Mod	Veri	Başlatma	Amaç
Development	`DATABASE_URL` yoksa in-memory; varsa PostgreSQL	`npm run dev`	Hızlı geliştirme, kolay test ve watcher
Playwright	İzole in-memory backend	Playwright webServer	Yerel PostgreSQL'i kirletmeden tarayıcı testi
Docker Compose	PostgreSQL volume + migration	`docker compose up -d --build`	Sunuma ve release'e yakın bütünlük ortamı

4. Teknoloji Seçimleri ve Gerekçeleri

Teknoloji yığını, gerçek zamanlı kurumsal mesajlaşma, ekipteki dil dağılımı ve iki haftalık demo süresinde güvenilir sonuç üretme ihtiyacına göre şekillenmiştir.

Teknoloji	Kullanıldığı yer	Seçilme nedeni
TypeScript	Frontend ve backend	Aynı dil ile uçtan uca geliştirme, DTO ve domain şekillerinde tip güvenliği, refactor kolaylığı.
React 18	Frontend	Bileşen tabanlı dashboard, zengin ekosistem ve sohbet ekranındaki yoğun etkileşimleri yönetme.

Teknoloji	Kullanıldığı yer	Seçilme nedeni
Redux + Redux-Saga	Frontend state	Sohbet listeleri, seçili konuşma, auth, profile ve async API işlerini merkezi ve izlenebilir biçimde yönetme.
Bootstrap 5 + Reactstrap + SCSS	Arayüz	Hazır responsive grid/components, hızlı demo üretimi ve tema özelleştirme.
Axios	REST client	Interceptor tabanlı JWT header, hata dönüşümü ve ortak API istemcisi.
NestJS 11	Backend	Modüler DI yapısı; guard, pipe, interceptor, filter ve Swagger entegrasyonunun düzenli biçimde sunulması.
PostgreSQL 16	Veritabanı	Kullanıcı/konuşma/rol/ticket ilişkilerinde ACID, foreign key, indeks ve transaction desteği.
Prisma 6	ORM + migration	Tip üreten client, açık schema, tekrarlanabilir migration ve SQL'e gerektiğinde doğrudan erişim.
Socket.IO 4	Realtime	Room modeli, ACK, otomatik reconnect, browser uyumluluğu ve ham WebSocket'e göre daha kısa geliştirme süresi.
WebRTC	Sesli arama	Ses medyasını sunucuda taşımadan veya depolamadan peer-to-peer görüşme; Socket.IO yalnız sinyalleme yapar.
Java 17 + Spring Boot 4	Webhook adaptörü	Java teslim gereksinimini bağımsız servis sınırında karşılamak, validation ve güvenilir HTTP client davranışı.
Docker Compose	Çalıştırma	PostgreSQL, migration, bootstrap, backend, Java ve frontend'i aynı sırayla tekrarlanabilir başlatma.
Nginx unprivileged	Frontend runtime	Statik React build'ini küçük ve non-root image ile sunmak; API/WebSocket same-origin proxy.
Swagger/OpenAPI	API keşfi	Backend, frontend ve dış entegrasyon ekiplerinin kontratı görmesi ve demo sırasında canlı test.
Jest/Supertest/Playwright/JUnit	Kalite	Unit, HTTP e2e, gerçek tarayıcı ve Java servis testlerini katmanlarına uygun araçlarla yapmak.

4.1 Bilinçli mimari tercihlerin sonucu

- Socket.IO, REST'in yerine geçmez: kalıcı state değişiklikleri servis katmanında ortaklaştırılır; REST ve socket aynı iş kurallarını kullanır.
- WebRTC yalnız direct conversation için açılır; grup araması gibi ayrı bir SFU/medya sunucusu gerektiren kapsam demo dışında tutulur.
- Java servisi ana backend'e gömülmez; dış ticket sözleşmesini çeviren dar bir adapter olarak kalır.
- PostgreSQL production'da zorunludur; in-memory mod yalnız geliştirme ve izolasyon testi kolaylığı sağlar.

5. Depo ve Kod Organizasyonu

Monorepo yapısı, üç çalıştırılabilir uygulamayı ve ortak operasyon dokümanlarını tek Git deposunda tutar.

Yol	Sorumluluk
`backend/src`	NestJS modülleri, controller/service/guard/gateway ve ortak request pipeline
`backend/prisma`	PostgreSQL model şeması ve 13 sıralı migration
`backend/scripts`	Smoke/load/full-stack test, migration audit, backup/restore ve bootstrap araçları
`backend/test`	Supertest tabanlı HTTP e2e senaryoları
`frontend/src/api`	REST ve Socket.IO istemcileri ile backend response adapter'ları
`frontend/src/redux`	Auth, chats, calls, contacts, bookmarks, profile, settings ve layout state/saga'ları
`frontend/src/pages`	Authentication ve dashboard ekranları
`frontend/src/features`	WebRTC audio call ve presence provider gibi çapraz özellikler
`frontend/e2e`	33 gerçek tarayıcı senaryosunu içeren Playwright paketi
`java-webhook/src`	Spring Boot webhook adapter'ı ve JUnit testleri
`docs`	Backend kontratı, database runbook, release notları, Postman ve handoff
`docker-compose.yml`	Bütün servislerin başlatma sırası, portları, healthcheck ve env bağlantıları

5.1 Backend modül sınırları

Modül	Sorumluluk
Auth	Register/login, JWT üretimi, current user, password change
Users	Kullanıcı oluşturma, arama, profil, automation ID ve bot hesabı
Conversations	Direct/group/management conversation, mesaj, attachment, üye ve preference kuralları
Chat	Socket.IO auth, rooms, presence, typing, realtime mesaj ve WebRTC signaling
Calls	Kalıcı çağrı kayıtları ve kullanıcıya göre call history
Bookmarks	Kullanıcıya özel mesaj bookmark CRUD
Contact Invitations	Davet gönderme, kabul/red ve direct contact oluşturma
Tickets	Ticket havuzu, atama, optimistic version ve activity history
Bot	Shared-secret korumalı dış otomasyon grubu işlemleri
Metrics	Admin'e özel HTTP/socket/message sayaçları
Health	Liveness endpoint'i
Dev	Yalnız development için kontrollü veri reset endpoint'i
Database	Prisma yaşam döngüsü ve opsiyonel in-memory/persistent çalışma

6. Backend Derin İnceleme

NestJS backend, controller'ları yalnız HTTP sözleşmesi için kullanır; esas domain kuralları servislerde tutulur. Böylece REST ve Socket.IO aynı servis metodlarını çağırabilir.

6.1 Global request pipeline

- Request logger güvenli bir `x-request-id` üretir veya kabul eder; hassas query parametrelerini `REDACTED` yapar.
- CORS allowlist ve Socket.IO origin ayarı aynı kaynaktan okunur.
- Helmet güvenlik header'larını, compression cevap sıkıştırırmayı uygular.
- Global ValidationPipe yalnız DTO'da tanımlı alanları kabul eder; bilinmeyen alanlar reddedilir.
- JWT ve Roles guard kimlik/rol kontrolünü yapar; conversation servisleri ayrıca üyelik ve grup rolünü kontrol eder.
- ResponseInterceptor başarılı yanıtları standart `{ success, data }` zarfına alır.
- HttpExceptionFilter hataları `{ success:false, statusCode, message, error, timestamp }` biçimine getirir.

6.2 Kimlik doğrulama

- Parolalar bcrypt cost 10 ile hashlenir; düz parola veritabanında tutulmaz.
- JWT payload içinde kullanıcı ID (`sub`), e-posta ve global rol bulunur; varsayılan süre 1 gündür.
- Bearer token HTTP header'dan, Socket.IO token ise handshake auth/header alanından alınır.
- Login dakikada 5, register dakikada 10 istekle sınırlandırılmıştır; frontend 429 durumunda Retry-After geri sayımı gösterir.
- Şifre değiştirme mevcut parolayı tekrar doğrular ve yeni hash'i kaydeder.

6.3 Conversation kuralları

- Aynı iki kullanıcı için direct conversation idempotenttir; tekrar create çağrısı yeni kayıt üretmez.
- Automation bot ile normal direct chat engellenir.
- Grup oluşturma global admin ister. Oluşturan kişi owner olur; manager ve member listeleri DTO ile atanabilir.
- Owner/manager/member yetkileri servis katmanında kontrol edilir. Owner sahipliği aktif bir üyeye devredebilir.
- Manager chat, ana gruba birebir bağlı `management` conversation'dır ve yalnız owner/manager rollerine görünür.

- `memberCanSendMessages`, `membersCanLeave` ve `status` alanları mesaj/ayrılma davranışını belirler.
- Archive, bookmark ve delete konuşmayı global olarak değiştirmez; `ConversationPreference` ile kullanıcıya özeldir.

6.4 Mesaj güvenilirliği

- `clientMessageld` + `senderId` unique kuralı retry sırasında aynı mesajın iki kez yazılmasını önler.
- Socket bağlıysa ACK beklenir; bağlantı yoksa frontend aynı `clientMessageld` ile REST fallback yapar.
- Mesaj düzenleme ve soft delete yalnız gönderene açıktır; sistem mesajları kullanıcı tarafından değiştirilemez.
- Reply hedefi aynı conversation içinde ve erişilebilir olmalıdır; silinen hedefte ilişki `SetNull` olur.
- Forward edilen mesajlar kalıcı `isForwarded` alanıyla etiketlenir; attachment'lar hedef conversation'a yeniden yüklenir.
- Pagination `before` cursor ile geriye doğru çalışır; arama conversation üyeliğini kontrol eder.

6.5 Attachment doğrulaması

Kural	Değer
Mesaj başına dosya	En fazla 5
Dosya başına boyut	En fazla 5 MB
İzin verilen image	JPEG, PNG, GIF, WebP
İzin verilen belge	PDF, text/plain
İzin verilen audio	MPEG, WAV, OGG, WebM, MP4
Güvenlik	MIME allowlist + dosya imzası kontrolü + normalize edilmiş dosya adı
Erişim	Download yalnız aktif conversation katılımcısına
Saklama	Binary içerik PostgreSQL `Bytes` alanında

7. Veritabanı Tasarımı

PostgreSQL şeması, mesajlaşma ile ticket/arama özelliklerini aynı ilişki bütünlüğü içinde tutar. Prisma schema kaynak gerçek, migration'lar ise değişiklik geçmiştir.

Model	Temel alanlar	Tasarım amacı
User	Kimlik, automationId, email, username, passwordHash, rol ve profil	Conversation, message, ticket, invitation, bookmark ve call ilişkilerinin merkezi
Conversation	direct/group/management, isim, politika, durum, externalRef	Ana sohbet nesnesi; management chat self-relation ile bağlı
ConversationParticipant	conversationId + userId, owner/manager/member, read/leave zamanları	Composite primary key; üyelik ve read state
Message	sender, content, type, clientMessageld, reply target, forwarded, zamanlar	Soft delete, reply self-relation ve retry idempotency
MessageAttachment	Dosya adı, MIME, boyut ve binary data	Mesaj silinince cascade
MessageBookmark	userId + messageld, isteğe bağlı başlık	Kullanıcıya özel composite key
ConversationPreference	bookmark/archive/delete bayrakları	Conversation görünümünü kullanıcı bazında ayırır
ContactInvitation	Gönderen, alıcı, mesaj, pending/accepted/declined	Kabul edildiğinde direct conversation ilişkisi oluşur
SupportTicket	Konu, mesaj, priority, status, assignee, note, version	Çoklu admin ticket havuzu ve optimistic concurrency
SupportTicketActivity	Actor, action, önceki/yeni değer ve zaman	Ticket işlem geçmişi
CallRecord	Caller, recipient, status, start/answer/end ve reason	Direct WebRTC görüşmelerinin kalıcı özeti

7.1 Enum sözlüğü

Enum	Değerler
UserRole	admin, user
ConversationType	direct, group, management
ParticipantRole	owner, manager, member
ConversationStatus	active, closed, archived
MessageType	user, system
ContactInvitationStatus	pending, accepted, declined
CallStatus	ringing, active, completed, missed, declined, failed
SupportTicketPriority	low, medium, high
SupportTicketStatus	open, in_progress, resolved, closed
SupportTicketActivityAction	created, assigned, unassigned, transferred, status_changed, priority_changed, note_updated

7.2 İlişki ve silme kararları

- Conversation silinirse participant, message, preference ve call kayıtları cascade edilir.
- User silinmesi participant ve kişisel alt kayıtları cascade eder; message sender ve call endedBy alanı `SetNull` ile tarihsel kaydı korur.
- Conversation creator `Restrict` kullanır; oluşturucu varken konuşmanın yetim kalması engellenir.
- Management conversation ana gruba unique self-relation ile bağlıdır ve ana grup silinince cascade olur.
- Mesaj reply hedefi silinirse referans `SetNull`; cevap mesajının kendisi korunur.

7.3 İndeks stratejisi

İndeksler, en sık sorgulanan kullanıcı konuşmaları, conversation mesaj zaman sırası, ticket durum/atama filtreleri, pending davetler, bookmark/preference bayrakları ve call history yönleri üzerine kurulmuştur. `database-audit.js` migration sonrası beklenen indeks ve foreign key delete kurallarını otomatik doğrular.

8. REST API Referansı

Tüm HTTP endpoint'leri `/api` global prefix'i altındadır. Bot endpoint'leri hariç korumalı endpoint'ler Bearer JWT kullanır.

Alan	Metot	Yol	Amaç
Auth	POST	/auth/register	Yeni user kaydı ve JWT
Auth	POST	/auth/login	E-posta/parola ile JWT
Auth	GET	/auth/me	Aktif kullanıcı
Auth	PATCH	/auth/password	Mevcut parolayı doğrulayıp değiştirme
Users	GET	/users?search=	Kullanıcı arama/listeleme
Users	GET	/users/me	Kendi profili
Users	PATCH	/users/me	About, location ve profile image güncelleme
Users	GET	/users/:userId	Kullanıcı profili
Conversations	POST	/conversations/direct	Idempotent direct conversation
Conversations	POST	/conversations/groups	Admin ile grup oluşturma
Conversations	GET	/conversations	Kullanıcının konuşmaları ve filtreler
Conversations	GET	/conversations/contacts	Active direct ilişkilerden kişi listesi
Conversations	GET	/conversations/:id	Conversation detayı
Preferences	PATCH	/conversations/:id/bookmark	Conversation favorite toggle
Preferences	PATCH	/conversations/:id/archive	Kullanıcıya özel archive toggle

Alan	Metot	Yol	Amaç
Preferences	DELETE	/conversations/:id	Kullanıcıya özel conversation gizleme
Groups	GET	/conversations/:id/management	Yetkili roller için özel manager chat
Groups	PATCH	/conversations/:id	İsim, açıklama, politika ve durum
Groups	PATCH	/conversations/:id/owner	Owner transfer
Messages	POST	/conversations/:id/messages	Text/reply/forwarded mesaj
Messages	POST	/conversations/:id/messages/attachments	Multipart kalıcı attachment
Messages	GET	/conversations/:id/attachments/:attachmentId	Yetkili download/inline
Messages	GET	/conversations/:id/messages	Cursor pagination
Messages	GET	/conversations/:id/messages/search	Conversation içi arama
Messages	PATCH	/conversations/:id/messages/:messageId	Gönderenin mesajını düzenleme
Messages	DELETE	/conversations/:id/messages/:messageId	Gönderenin mesajını soft delete
Read state	PATCH	/conversations/:id/read	Conversation'ı okundu yapma
Read state	PATCH	/conversations/:id/messages/:messageId/unread	Seçili mesajdan itibaren okunmamış
Groups	POST	/conversations/:id/leave	Politikaya uygun gruptan ayrılma
Groups	GET	/conversations/:id/participants	Aktif katılımcılar
Groups	POST	/conversations/:id/participants	Yetkili üye ekleme
Groups	DELETE	/conversations/:id/participants/:userId	Yetkili üye çıkarma
Groups	PATCH	/conversations/:id/participants/:userId/role	Manager/member rolü
Bookmarks	GET	/bookmarks	Kendi kayıtlı mesajları
Bookmarks	POST	/bookmarks	Mesaj kaydetme
Bookmarks	PATCH	/bookmarks/:messageId	Bookmark başlığı
Bookmarks	DELETE	/bookmarks/:messageId	Bookmark kaldırma
Invitations	POST	/contact-invitations	Kişi daveti
Invitations	GET	/contact-invitations	Gelen pending davetler
Invitations	PATCH	/contact-invitations/:id	Kabul veya red
Calls	GET	/calls	Kullanıcının son sesli çağrıları
Tickets	POST	/tickets	Destek talebi oluşturma
Tickets	GET	/tickets	Role göre görünür ticket listesi
Tickets	GET	/tickets/:id	Ticket detayı ve activity
Tickets	PATCH	/tickets/:id	Atanan admin ticket güncellemesi
Tickets	POST	/tickets/:id/claim	Admin ticket üzerine alma
Tickets	PATCH	/tickets/:id/assignee	Atama, transfer veya havuza bırakma
Bot	POST	/bot/groups	externalRef ile idempotent bot grubu
Bot	POST	/bot/create-group	Java webhook için legacy alias
Bot	POST	/bot/groups/:id/participants	Automation group üyeleri
Bot	POST	/bot/groups/:id/messages	Bot adına kalıcı mesaj
Bot	PATCH	/bot/groups/:id	Bot grup politikası/durumu
Bot	PATCH	/bot/groups/:id/participants/:userId/role	Bot manager atama
Metrics	GET	/metrics	Yalnız admin runtime sayaçları
Health	GET	/health	Backend liveness
Dev	POST	/dev/reset	Yalnız açık development ortamında kontrollü reset

8.1 Standart response ve hata biçimi

```
// Başarılı
{
  "success": true,
  "data": { ... }
}

// Hata
{
  "success": false,
  "statusCode": 401,
  "message": "Unauthorized",
  "error": "UNAUTHORIZED",
  "timestamp": "2026-07-27T..."
}
```

9. Socket.IO ve Gerçek Zamanlı Akış

Socket.IO namespace'i `/chat` altında JWT ile açılır. Her socket kullanıcı odasına, yetkili olduğu conversation odalarına katılır. Sunucu hem room hem user-room yayınlarını kullanarak açık ekran ve liste state'ini senkron tutar.

9.1 İstemciden sunucuya event'ler

Event	Davranış
presence:sync	Kişi çevrimiçi snapshot'ını ister
conversation:join	Üyelik kontrolü sonrası conversation room'a katılır
conversation:sync	Reconnect sonrası conversation/message farklarını eşitler
conversation:unsubscribe	Room dinlemeyi bırakır; üyelik silmez
conversation:leave	Grup ayrılma iş kuralını çalıştırır
conversation:update	Grup ayarlarını yetkiyle değiştirir
conversation:transfer-owner	Sahipliği aktif üyeye devreder
message:send	ACK'li, idempotent mesaj gönderir
message:update / message:delete	Gönderenin mesajını günceller veya siler
typing:start / typing:stop	Geçici typing state yayınlar
message:read	Read receipt ve lastReadAt günceller
call:start / accept / reject	Direct çağrı oturum yaşam döngüsü
call:signal	SDP/ICE sinyalinin yalnız karşı tarafa taşır
call:sync / recover / end	Reconnect, ICE recovery ve çağrı bitirme

9.2 Sunucudan istemciye event'ler

Event grubu	İstemci etkisi
session:ready	Socket doğrulandı; user ve conversation kimlikleri hazır
presence:contacts / snapshot	Toplu online state
presence:online / offline	Aktif socket sayısına göre değişim
conversation:created / joined / synced / updated / left	Conversation lifecycle
participant:added / removed / left	Üyelik ve rol değişiklikleri
message:new / updated / deleted / read	Kalıcı mesaj state'i
typing:started / stopped	Geçici yazıyor göstergesi
contact:invitation:new / updated	Davetlerin realtime görünmesi
call:incoming / accepted / rejected / ended	Çağrı UI state'i
call:signal / recovery-needed / history-updated	WebRTC signaling ve kalıcı history refresh
exception	Kontrollü socket hata kodu ve mesajı

9.3 Presence ve reconnect

Presence tek socket'e değil, kullanıcı başına aktif socket kümesine dayanır. Böylece aynı kullanıcı iki sekme açtığında bir sekmenin kapanması kullanıcıyı yanlışlıkla offline yapmaz. Reconnect sırasında `conversation:sync` kaçırılan state'i, `call:sync` devam eden çağrıyı geri yükler. Çağrılarda 15 saniyelik disconnect grace period kısa ağ kesintilerinde oturumun erken kapanmasını önler.

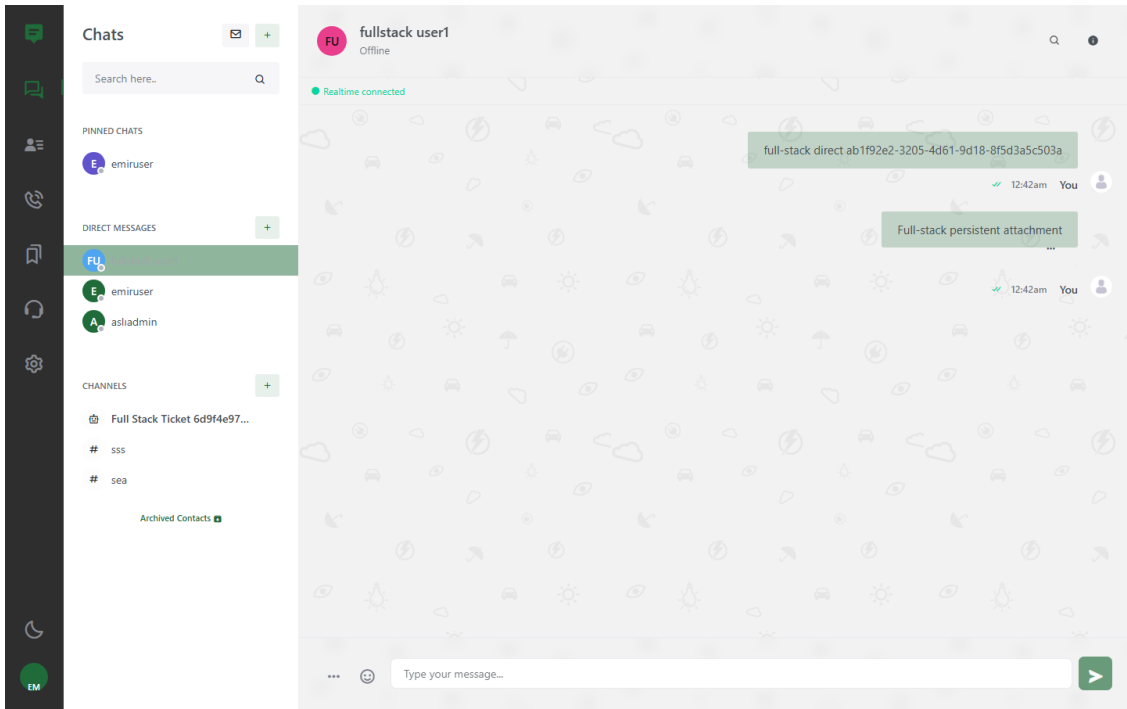
10. Frontend Mimarisi

React uygulaması yoğun sohbet state'ini Redux/Saga ile, WebRTC ve presence gibi yaşam döngüsü ağır özellikleri Provider bileşenleriyle yönetir.

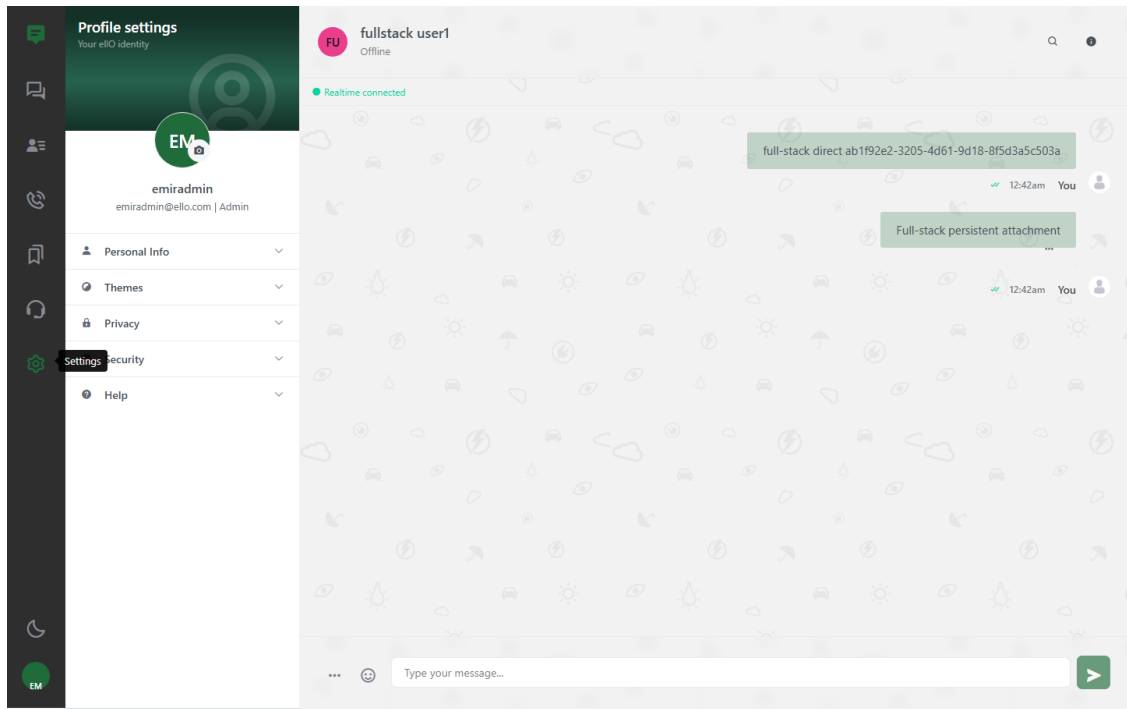
Katman	Sorumluluk
Routes/AuthProtected	Public/private route ve geçersiz session temizleme
APIClient	Base URL, Bearer header, response envelope açma ve hata normalizasyonu
Backend adapters	Backend user/conversation/message modelini tema UI modeline dönüştürme
Redux reducers	Auth, chat listeleri, seçili chat, bookmarks, calls, profile, settings ve layout state
Redux-Saga	API side effect, retry/fallback ve state update akışları
PresenceProvider	Socket presence sync ve kullanıcı online state
AudioCallProvider	RTCPeerConnection, media stream, signaling, reconnect ve overlay
Pages/Components	Dashboard, composer, message row, management, support ve settings UI

10.1 Dashboard navigasyonu

- Chats: pinned/favorite, direct messages, channels ve archived contacts.
- Contacts: direct ilişkilere göre kişiler, arama ve davetler.
- Calls: gelen/giden/tamamlanan/reddedilen/cevapsız çağrı geçmişi ve tekrar arama.
- Saved Messages: kullanıcıya özel bookmark listesi ve mesaja geri dönme.
- Support: user ticket görünümü ve admin ticket havuzu.
- Settings: profil, tema, privacy/security seçenekleri ve help.
- Profile: kullanıcının kendi profili ve bağlı grupları.



Şekil 3 - Çalışan Compose ortamında direct sohbet ve realtime bağlantı göstergesi.



Şekil 4 - Profil ve ayarlar paneli.

11. Uygulamanın Özellikleri

Bu bölüm kullanıcı tarafından görülen özellikleri ve bunların backend karşılıklarını bir arada açıklar.

11.1 Authentication ve oturum

- Register, login, logout, protected route ve current-user doğrulaması.
- JWT localStorage oturumu; sayfa yenilemesinde session korunur, geçersiz token dashboard açılmadan temizlenir.
- Admin/user rolüne göre grup oluşturma ve ticket ekranı farklılaşır.
- Şifre değiştirme gerçek backend endpoint'ine bağlıdır.
- Login rate limit cevabı kullanıcıya anlaşılır hata ve geri sayım olarak gösterilir.

11.2 Direct mesajlaşma ve Contacts

- Bir kullanıcı seçildiğinde var olan direct conversation bulunur veya tek kez oluşturulur.
- Contacts ayrı bir tablo değil, aktif direct conversation katılımcılarından türetilir; duplicate ilişki önlenir.
- Kullanıcı kendi ekranında sohbeti archive/delete yapsa bile direct ilişki contact picker için korunur.
- Contact invitation gönderilebilir; kabul edildiğinde direct conversation açılır, duplicate pending davet ve mevcut contact durumu reddedilir.
- Profile panelinden bookmark/archive/delete ve direct konuşma açma işlemleri yapılabilir.

11.3 Grup ve channel yönetimi

- Admin grup adı, açıklama, üyeler ve manager'lar ile grup oluşturur.
- Grup owner'ı, manager'ı ve member'ı ayrı yetkilere sahiptir.
- Grup adı/açıklaması, active/closed/archived durumu, member mesaj ve ayrılma politikası değiştirilebilir.
- Owner aktif üyeye sahiplik devreder; manager/member rolü atanabilir.
- Üye ekleme/çıkarma/ayırılma tüm açık istemcilere realtime yansır.
- Owner ve manager'lar ana gruba bağlı özel management conversation görür; normal üyeler erişemez.
- Üye profile action'ı geçerli direct conversation açar; contact ID ile conversation ID karışıklığı giderilmiştir.

11.4 Mesaj özellikleri

Özellik	Uygulama davranışı
Text mesaj	REST veya Socket.IO ACK; kalıcı ve idempotent
Reply	Hedef mesaj DB ilişkisiyle korunur; refresh sonrası iki kullanıcıda görünür
Forward	Metin ve attachment hedef chat'e kopyalanır; `Forwarded` etiketi kalıcıdır
Copy	Mesaj metni Clipboard API ile kopyalanır
Mark unread	Seçili mesaj zamanından hemen önce `lastReadAt` ayarlanır ve badge oluşur
Edit	Yalnız mesaj sahibi; realtime güncelleme
Delete	Yalnız mesaj sahibi; soft delete ve realtime yansıma
Search	Conversation içinde sunucu tarafı arama; sonuca tıklayınca mesaj odaklanır
Bookmark	Kullanıcıya özel kaydetme, başlık güncelleme ve Saved Messages'tan geri açma
Attachment	Image/PDF/text/audio upload, authenticated preview/download
Shared contact	Composer'dan yapılandırılmış contact mesajı
Camera capture	Tarayıcı kamera/natif file input ile görsel ekleme
Emoji	Emoji picker ile composer'a içerik
Typing/read	Socket event'i ile typing indicator ve read receipt

11.5 Conversation listesi

- Direct ve group listeleri ayrı yüklenir; tekrarlanan istekler promise birleştirme ile azaltılır.
- Pinned/favorite, archive ve kullanıcıya özel delete tercihleri PostgreSQL'de kalıcıdır.
- Unread badge, açık konuşmaya gelen mesaj ve hidden-tab title notification realtime güncellenir.
- Incoming message popup/toast kaldırılmıştır; çift bildirim gürültüsü önlenmiştir.
- Conversation içinde mesaj arama ve Saved Messages'tan hedef mesaja scroll yapılır; scroll yalnız sohbet container'ını hareket ettirir.

11.6 Birebir sesli arama

- 1 Arayan, aktif direct conversation içindeki karşı kullanıcı için `call:start` gönderir.
- 2 Backend iki tarafın üyeliğini, bot olmadığını ve ulaşılabilirliği kontrol eder; CallRecord oluşturur.
- 3 Alıcı `call:incoming` overlay'i görür; kabul veya red verir.
- 4 Kabulden sonra SDP offer/answer ve ICE candidate'lar Socket.IO ile yalnız hedef kullanıcıya relay edilir.
- 5 Ses `getUserMedia` ve RTCPeerConnection üzerinden peer-to-peer akar; mute/unmute istemci tarafındadır.
- 6 End/reject/missed/failed sonucu PostgreSQL call history'ye yazılır ve Calls sekmesi güncellenir.
- 7 Geçici socket kesintisinde 15 saniye grace period, `call:sync`, ICE restart ve `call:recover` ile görüşme kurtarılmaya çalışılır.

WebRTC kapsamı

Mevcut özellik yalnız birebir sesli aramadır. Farklı ağ/NAT kombinasyonlarında production güvenilirliği için kimlik doğrulama TURN gerekir. VideoCallModal dosyası temadan kalmış olsa da çalışan video arama akışı yoktur.

11.7 Support ticket yönetimi

- User konu, mesaj ve öncelikle ticket oluşturur; yalnız kendi taleplerini görür.
- Admin tüm taleplerde arama, durum, öncelik ve assignment filtresi kullanır.
- Unassigned ticket admin tarafından claim edilebilir; başka admin'e transfer veya havuza bırakılabilir.
- Yalnız atanmış admin ticket durumunu, önceliğini ve admin notunu günceller.
- Her değişiklik actor ve zaman bilgisiyle SupportTicketActivity'ye yazılır.
- Optimistic `version` eski ekrandan yapılan çakışan güncellemeyi 409 Conflict ile reddeder.

11.8 Profil, tema ve yardım

- Username, email, rol, about, location ve profile image gösterilir; about/location/image backend'e kaydedilir.
- Profile image browser'da sıkıştırılır, güvenli data URL tür/boyut kontrolünden sonra saklanır.
- Light/dark görünüm localStorage'a yazılır ve sayfa yenilemesinde korunur.
- Help alanında FAQ, Contact ve Terms & Privacy modalları çalışır.
- Privacy, security notification, tema rengi ve arka plan deseni kontrolleri arayüzde vardır ancak sunucuya kalıcı ayar olarak yazılmaz; bu durum roadmap kapsamındadır.

11.9 Responsive ve erişilebilirlik

- Mobil sohbet alanı `100dvh` ve safe-area tamponlarıyla composer'ın telefon gezinme alanına çarpmasını önler.
- Chat ve group info panelleri 390x844 viewport'ta horizontal overflow olmadan Playwright ile kontrol edilir.
- Ana menü icon butonları aria-label ve tooltip taşır.
- Login ve dashboard için axe tabanlı ciddi accessibility ihlali testi vardır.

12. Java Webhook Servisi

Java servisi, dış ticket olayını ana backend domain sözleşmesine çeviren bağımsız ve dar sorumluluklu bir adapter'dır.

12.1 Akış

- 1 Dış sistem `POST /webhook/ticket-created` isteğini `X-Webhook-Token` ile gönderir.
- 2 Spring Validation eventType, ticketId, ownerId, title ve participantIds alanlarını doğrular.
- 3 Webhook token sabit zamanlı `MessageDigest.isEqual` karşılaştırmasıyla kontrol edilir.
- 4 WebhookForwardingService payload'ı CreateBotGroupRequest biçimine dönüştürür.

- NestChatClient, `x-bot-secret` header ile `/api/bot/create-group` endpoint'ine gider.
- Backend `externalRef=ticketId` sayesinde tekrar webhook'larda aynı grubu döndürür.
- Backend 4xx verirse retry yapılmaz; connection/5xx durumunda konfigüre limit içinde retry yapılır; son hata 502'ye çevrilir.

12.2 Health ve runtime

Bileşen	Davranış
GET /health	Java process liveness; backend'i kontrol etmez
GET /ready	NestJS `/api/health` bağımlılığını kontrol eder; başarısızsa 503
Timeout	Varsayılan connect 1s, read 3s
Retry	Varsayılan 2, en fazla 3 deneme; delay varsayılan 100ms
Container	Multi-stage Maven build, non-root `ello` kullanıcısı, JRE-only runtime

POST /webhook/ticket-created
X-Webhook-Token: <WEBHOOK_SECRET>

```
{
  "eventType": "ticket.created",
  "ticketId": "TICKET-42",
  "ownerId": "00000000-0000-4000-8000-000000000001",
  "title": "Support Room",
  "participantIds": ["00000000-0000-4000-8000-000000000002"]
}
```

13. Bot Otomasyonu

Bot API, normal kullanıcı JWT'sinden ayrı bir güven alanıdır ve tüm endpoint'lerde `x-bot-secret` kullanır.

- Automation bot, sistem tarafından oluşturulan ve normal direct message listesinden gizlenen özel user kayıdır.
- Bot group create ownerId, participantIds, managerIds, name, externalRef ve initialMessage kabul eder.
- Built-in kullanıcılar UUID yerine kısa automation ID ile referanslanabilir.
- `externalRef` unique ve normalize edilir; aynı referansla eşzamanlı retry tek grup üretir.
- İlk yanıt `created:true, reused:false`; tekrar yanıt `created:false, reused:true` taşır.
- Retry payload'ındaki farklı alanlar mevcut grubu sessizce değiştirmez; mutation endpoint'i kullanılmalıdır.
- Bot grupları `isBotManaged` ve `sourceName` ile işaretlenir; politika gereği normal üyeler read-only olabilir.

13.1 Ticket'tan gruba örnek

Ticket alanı	Bot grup alanı	Amaç
ticketId	externalRef	Webhook retry idempotency
ownerId	ownerId	Grup owner/manager bağlantı
participantIds	participantIds	İlgili kullanıcıları gruba ekleme
title	name	Kullanıcıya görünen channel adı
ticket created	initialMessage	Grup içinde başlangıç sistem bağlantı

14. Güvenlik ve Dayanıklılık

Demo ürünü olmasına rağmen backend'de kimlik, yetki, input, secret, rate limit, attachment ve operasyon sınırları uygulanmıştır.

Alan	Uygulanan kontrol
Authentication	JWT expiration açık; bearer doğrulaması; bcrypt password hash

Alan	Uygulanan kontrol
Authorization	Global role guard + conversation membership + group role matrisi
Input	class-validator whitelist, bilinmeyen alanları reddetme, UUID pipe
HTTP rate limit	Global 120/60s; login 5/60s; register 10/60s; bot endpoint'e özel limitler
Socket rate limit	Socket başına 60 event/10s bucket ve disconnect cleanup
CORS	Production wildcard yasak; allowlist HTTP ve Socket.IO için ortak
Headers	Helmet, compression, credential/exposed header kontrolü
Secrets	Production secret en az 32 rastgele karakter; placeholder ve local-compose kalıpları reddedilir
Bot/webhook secret	Timing-safe karşılaştırma; source/response loglarına yazılmaz
Logging	Request ID; password/token/secret query parametrelerini redaction; SDP/ICE içeriğini loglamama
Attachments	MIME + signature + boyut + sayı + participant-only download
Production flags	Demo users/dev reset/backend demo UI production profilinde kapalı
Container	Backend, frontend ve Java runtime non-root; multi-stage build
Port exposure	PostgreSQL, backend ve Java yalnız 127.0.0.1; frontend 5173 dışı açık

14.1 Bilinen güvenlik borcu

- Frontend JWT'yi localStorage'da tutar; gelecekte XSS etkisini azaltmak için HttpOnly cookie veya daha güçlü token mimarisi değerlendirilebilir.
- Parola değişikliği mevcut JWT'leri toplu olarak revoke etmez; token version/session table yoktur.
- Nginx katmanında uygulamaya özel CSP, Permissions-Policy ve frame politikası henüz tanımlı değildir.
- Demo hesap şifreleri zayıftır ve bilinçli olarak sabittir.
- Swagger demo kolaylığı için Compose'ta açıktır; gerçek production'da kapatılmalı veya erişimi sınırlandırılmalıdır.
- Frontend `react-scripts`/CRA zinciri modernizasyon ister; zorla audit fix uygulamak build'i bozabileceği için kontrollü migration gerekir.

15. Docker, Ağ ve Operasyon

Compose, yalnız container çalıştırmakla kalmaz; migration ve demo kullanıcı bootstrap sırasını health bağımlılıklarıyla yönetir.

Servis	Görev	Host portu	Başlatma bağımlılığı
postgres	PostgreSQL 16 + volume	127.0.0.1:5432	-
migrate	Prisma migrate deploy + DB audit	Yok	PostgreSQL healthy
built-in-users-bootstrap	Altı demo user + automation bot	Yok	Migration tamamlandı
backend	NestJS REST + Socket.IO	127.0.0.1:3000	Bootstrap tamamlandı
java-webhook	Spring Boot adapter	127.0.0.1:8080	Backend healthy
frontend	Nginx + React + proxy	0.0.0.0:5173	Backend ve Java healthy

15.1 Secret ve environment yönetimi

- Kök `.env` Git tarafından izlenmez; JWT_SECRET, BOT_WEBHOOK_SECRET ve WEBHOOK_SECRET burada tutulur.
- Compose gerekli secret eksikse başlamaz.
- Backend production başlangıcında placeholder, `change-me`, `replace-with` ve `local-compose-\*` secret'larını reddeder.
- Frontend API/Socket URL ve ICE server listesi build argument olarak verilir.
- PostgreSQL password ile DATABASE_URL parolası birlikte değiştirilmelidir.

15.2 Backup, restore ve recovery

`database-backup.js` custom-format `pg\_dump`, `database-restore.js` onay kilitli `pg\_restore` akışı sağlar. Recovery runbook, yedek alma, servisleri durdurma, restore, migration audit ve login doğrulama sırasını dokümante eder. Database Git ile taşınmaz; her makinenin Docker volume'u ayrıdır.

15.3 Çalıştırma

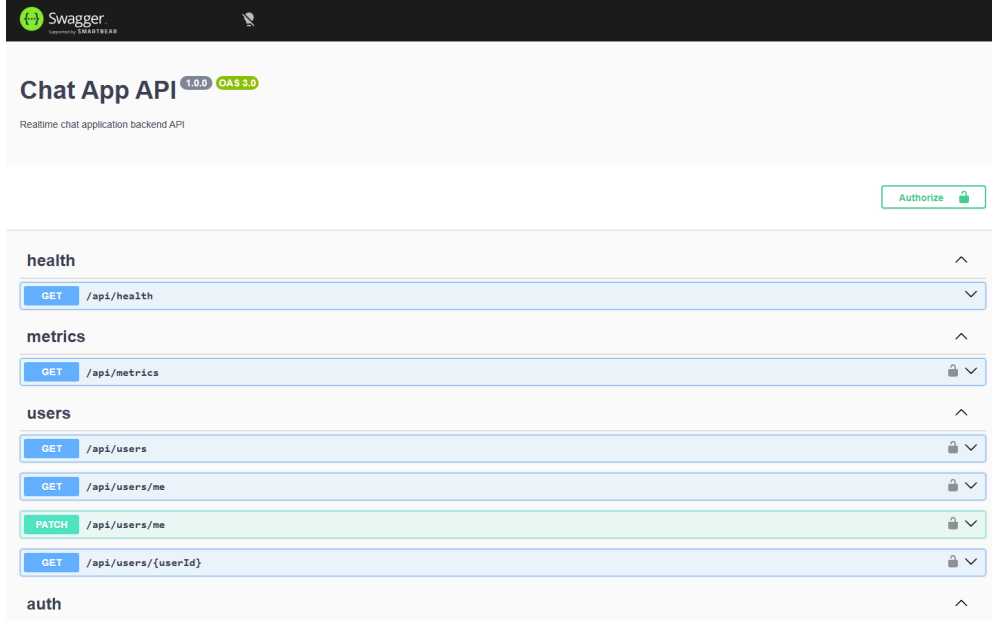
```
# İlk veya güncel build
docker compose up -d --build

# Durum
docker compose ps

# Uygulama
http://localhost:5173

# Swagger
http://localhost:3000/api/docs

# Tek komut bütünleşik test
npm.cmd run test:full-stack
```



Şekil 5 - Çalışan NestJS Swagger/OpenAPI arayüzü.

16. Test Stratejisi ve Kalite Güvencesi

Test piramidi, saf domain kurallarından gerçek tarayıcı ve Docker tabanlı release akışına kadar genişler.

Seviye	Araç	Kapsam
Backend unit	Jest	Env/CORS, rate limit, guard, request log, conversation, ticket, bookmark, invite, call ve gateway kuralları
Backend HTTP e2e	Supertest	Response envelope, DTO, auth matrix, retry, unread, reply, attachment, metrics, bot ve throttling
Frontend unit	Testing Library/Jest	Reducer, adapter, API error, call recovery, clientMessageld ve shared contact
Frontend E2E	Playwright	Gerçek browser login, roles, sohbet, group, presence, reply, forward, attachment, audio, mobile ve a11y
Java unit/integration	JUnit/Spring test	Validation, secret, timeout, retry, health/readiness, mapping ve 502
Full-stack	Node + Docker	PostgreSQL, migration, Nginx proxy, Swagger, roles, direct/group Socket.IO, webhook ve persistence
Database audit	Node + SQL metadata	Beklenen index ve foreign key/delete rule
Load test	Socket.IO clients	Ayarlanabilir socket ve mesaj yükü

16.1 Playwright'ta doğrulanan kullanıcı senaryoları

- Development hesap rolleri, protected route ve geçersiz session temizliği
- Tema kalıcılığı ve backend kapalıyken actionable login hatası
- Admin/user grup oluşturma yetki sınırı
- Bot grupları ve bot mesajlarının realtime görünmesi
- Direct-contact türetme, group picker ve invitation acceptance
- Profile, image, bookmark, archive ve delete
- Unread state, popup yokluğu, presence ve reconnect
- Search/bookmark/reopen/remove
- Image attachment upload, preview, copy, mark unread ve forward
- Composer action'ları ve media count görünümü
- Mobile overflow ve axe accessibility
- İki istemcide edit/delete/reply persistence
- Private manager chat ve group message realtime
- Audio answer/mute/end ve socket outage recovery
- Group management, leave ve member direct action

16.2 Son doğrulama özeti

Kontrol	Son kayıtlı sonuç
Backend unit	58/58 başarılı
Backend API e2e	13/13 başarılı
Frontend unit	18/18 başarılı
Playwright	33/33 başarılı
Full-stack Docker	Tüm 14 ana davranış grubu başarılı
Database audit	34 index ve 13 foreign key başarılı
Backend/frontend build	Production build başarılı
Manuel test	İki oturumda group realtime ve temel demo akışı başarılı

17. Geliştirme Sürecinde Neler Yapıldı ve Neden?

Geliştirme, önce kontrat ve temel backend, sonra tam yığın entegrasyon, ardından kalıcılık, güvenlik ve demo kullanılabilirliği şeklinde ilerlemiştir.

Tarih	Yapılan	Neden
13 Temmuz	NestJS temel backend, auth, users, conversation, message, read, search, Socket.IO, Swagger	Frontend/Java/DB ekiplerinin kullanacağı net bir backend kontratı oluşturmak
14 Temmuz	Demo UI, REST + socket mesaj akışı ve React frontend entegrasyonu	Backend davranışını gerçek kullanıcı ekranından gösterebilmek
15 Temmuz	HTTP/socket rate limit, CORS birlikteliği, kontrollü hata ve CI testleri	Realtime servisi kötü/gürültülü isteklerde dayanıklı yapmak
16 Temmuz	Demo users, PostgreSQL/Prisma ve Java webhook entegrasyonu	Ekip bağımlılıklarını birleştirmek ve kalıcı state'e geçmek
17 Temmuz	Login recovery UX, Contacts ve group management, persistence restart testi	Manuel testte çıkan kimlik/ID ve grup yetkisi sorunlarını kapatmak
20 Temmuz	Bot-managed group, manager chat, group policies ve profile iyileştirmeleri	Kurumsal otomasyon ve yönetici kontrollü channel davranışı sağlamak
21 Temmuz	DB backup/restore/audit, Java retry/readiness, production Compose, Nginx LAN proxy	Sunum ortamını tekrarlanabilir ve farklı cihazdan erişilebilir yapmak

Tarih	Yapılan	Neden
22 Temmuz	Support ticket, çoklu admin koordinasyonu, attachments, full-stack test	Kurumsal WhatsApp kapsamını destek operasyonuyla genişletmek
23-24 Temmuz	Invitations, search/bookmarks/preferences, WebRTC call history/presence	Kullanıcı deneyimini gerçek ürüne yaklaştırmak
24 Temmuz	Dört kullanıcı bug'ı: popup, contacts, bot idempotency ve audio reconnect	Manuel test geri bildirimlerini kapatıp demo stabilitesini yükseltmek
27 Temmuz	Reply/forward persistence, copy/unread, help, theme persistence, Compose secret/port hardening	Çalışmayan menüleri işler hale getirmek ve public demo riskini azaltmak
27 Temmuz	Swagger demo default'u ve full-stack admin varsayımı düzeltildi	Sunumda API testini kolaylaştırmak ve tek komut testin hazır hesaplarla çalışmasını sağlamak

17.1 Kapatılan dört kritik kullanıcı bug'ı

Bug	Problem	Çözüm
BUG-1	Aynı mesaj için popup gürültüsü	Incoming toast kaldırıldı; unread, list refresh ve tab title korundu
BUG-2	Direct chat ile Contact ilişkisi kopuk	Contact kaynağı active direct conversation olarak tekilleştirildi
BUG-3	Bot retry cevabı belirsiz	`created` ve `reused` metadata eklendi; farklı retry payload'ı mevcut grubu değiştirmiyor
BUG-4	Audio call socket kesintisinde erken bitiyor	15s grace, call sync/recover, ICE retry ve terminal event dedupe

17.2 Son demo işlev tamamlama paketi

- Reply artık yalnız görsel state değil, `replyToMessageId` DB ilişkisiyle kalıcıdır.
- Forwarded etiketi ve attachment forwarding refresh sonrası korunur.
- Message action menüsündeki Copy ve Mark unread gerçek API/browser davranışına bağlanmıştır.
- Settings Help içindeki FAQ, Contact ve Terms & Privacy butonları modal açar.
- Light/dark mode localStorage ile kalıcıdır.
- E2E senaryoları bu davranışları hem desktop hem mobil boyutta kapsayacak şekilde genişletilmiştir.

18. Tamamlanmamış veya Sınırlı Alanlar

Aşağıdaki maddeler çalışan özellik gibi sunulmamalıdır. Bunlar ürünün bir sonraki aşamasında tamamlanabilecek açık kapsam veya teknik borçtur.

Alan	Bugünkü durum	Tamamlama yolu
E-posta ile şifre sıfırlama	UI route/form var; API bilinçli olarak `not available yet` hatası döndürür	Mail provider, reset token tablosu, expiry ve tek kullanımlık token
Lock screen	Tema şablonu; gerçek session unlock davranışı yok	Aktif kullanıcı state'i ve backend re-auth
Video call	VideoCallModal dosyası var fakat hiçbir çalışan akışa bağlı değil	Video media track, UI ve ağ testi veya özelliğin kaldırılması
Privacy/Security ayarları	Kontroller görünür; update API şu anda no-op Promise	UserSettings modeli ve backend CRUD
Tema renk/desen	UI seçimi var, light/dark dışında kalıcı değil	LocalStorage veya backend preference
Şifre sonrası token iptali	Mevcut token süresi bitene kadar geçerli kalabilir	Session/tokenVersion ve revoke stratejisi
TURN altyapısı	Varsayılan public STUN; bazı NAT ağlarında bağlantı kurulamayabilir	Kimlik doğrulmalı TURN ve HTTPS deployment
Attachment ölçeklenmesi	Binary PostgreSQL'de; demo/orta hacim için uygun	Object storage + signed URL + malware scan
Horizontal scaling	Presence/call session process memory'sinde	Redis adapter, distributed presence ve shared call state
Frontend build zinciri	CRA/react-scripts eski ve audit borcu taşıyor	Vite/modern router migration ve bağımlılık temizliği
Production internet yayını	Compose yerel demo içindir; TLS ve edge güvenliği yok	Reverse proxy TLS, CSP, secret manager ve gerçek hesap politikası

Demo için kabul edilen risk

Sunum hedefi nedeniyle Swagger ve hazır test kullanıcıları varsayılan olarak açıktır. Bu karar kullanım kolaylığı içindir; production güvenlik varsayımı olarak yorumlanmamalıdır.

19. Önerilen Yol Haritası

Demo sonrasında ürünleştirme yapılacaksa öncelik, yeni özellik sayısından önce güvenli oturum ve production operasyonlarına verilmelidir.

Öncelik	Başlık	Öneri
P0	Demo freeze ve tekrar edilebilir sunum	Mevcut branch'i push/PR, demo checklist, backup ve tunnel kapatma prosedürü
P1	Oturum güvenliği	HttpOnly cookie veya session store, token revocation, güçlü gerçek kullanıcı şifreleri
P1	Edge güvenliği	HTTPS, CSP, Permissions-Policy, rate limit gözlemi ve secret manager
P1	Ayar kalıcılığı	Privacy/security/theme preference için UserSettings modeli ve API
P1	WebRTC production	TURN, HTTPS, farklı ağ matrisi ve çağrı kalite telemetrisi
P2	Dosya mimarisi	S3 uyumlu object storage, signed URL, thumbnail ve antivirus pipeline
P2	Realtime ölçek	Redis Socket.IO adapter, distributed presence/call session
P2	Auth özellikleri	Password reset, email verification, MFA ve session management
P2	Frontend modernizasyon	Vite, dependency audit, daha küçük bundle ve route-level lazy loading
P3	Ürün kapsamı	Push notifications, message reactions, group audio/video ve yönetim raporları

20. Demo ve Test Runbook

Aşağıdaki sıra, sunum öncesinde en az riskle sistemin hazır olduğunu doğrular.

20.1 Otomatik kontrol

```
cd C:\Users\WIN11\Documents\Staj
docker compose up -d --build
docker compose ps
npm.cmd run test:full-stack
```

20.2 Manuel iki oturum kontrolü

- Normal pencerede `emiradmin@ello.com / 123456` ile giriş yap.
- Gizli pencerede `emiruser@ello.com / 123456` ile giriş yap.
- Admin hesabından direct chat aç ve mesajın ikinci oturuma yenilemesiz geldiğini doğrula.
- Admin grup oluştur; user'ı ekle ve member mesaj politikasını açık tut.
- İki taraftan group mesajı, reply, forward ve mark unread dene.
- Bir image attachment gönder; preview ve download kontrol et.
- Direct conversation'da sesli arama başlat, cevapla, mute ve end akışını dene.
- User ticket oluştur; admin claim edip status/note değiştir.
- Swagger'da login, conversation list ve bot group idempotency örneğini göster.

20.3 Sunum sonrası

- Geçici Cloudflare tunnel açıldıysa container'ı durdur.
- DB'de sunum verisi korunacaksa backup al.
- Secret veya token ekran görüntüsü/log içine girdiyse rotate et.
- Manuel testte yeni hata varsa branch ve commit kimliğiyle bug kaydı oluştur.

Ek A. Veri Akışı Örnekleri

A.1 Direct mesaj

React composer

- > Socket.IO message:send (clientId)
- > ChatGateway authentication + rate limit
- > ConversationsService membership + policy
- > Prisma Message INSERT
- > RealtimeEventsService
- > message:new to conversation/user rooms
- > Redux conversation + unread refresh

A.2 Attachment

File input / camera

- > multipart POST /api/conversations/:id/messages/attachments
- > count/size/MIME/signature validation
- > Message + MessageAttachment transaction
- > message:new metadata
- > authenticated attachment GET for preview/download

A.3 Audio call

Caller call:start

- > backend membership/availability check
- > CallRecord(ringing)
- > call:incoming

Recipient call:accept

- > CallRecord(active)
- > SDP/ICE via call:sdp
- > peer-to-peer audio

call:end / timeout / disconnect grace expiry

- > persistent final CallStatus
- > call:history-updated

Ek B. Kaynak Kod Referans Haritası

Konu	Dosyalar
Uygulama başlangıcı	backend/src/main.ts, backend/src/config/configure-application.ts
Modül graph	backend/src/app.module.ts
Auth	backend/src/auth/*
Conversation domain	backend/src/conversations/conversations.service.ts
REST contract	backend/src/conversations/conversations.controller.ts
Realtime	backend/src/chat/chat.gateway.ts
Database	backend/prisma/schema.prisma ve backend/prisma/migrations/*
Frontend API	frontend/src/api/apiCore.ts, chats.ts, realtime.ts, backendAdapters.ts
Chat UI	frontend/src/pages/Dashboard/ConversationUser/*
Group management	frontend/src/pages/Dashboard/ConversationUser/GroupManagement.tsx
Audio	frontend/src/features/audio-call/*
Presence	frontend/src/features/presence/PresenceProvider.tsx
Support	backend/src/tickets/* ve frontend/src/pages/Dashboard/Support/index.tsx
Java adapter	java-webhook/src/main/java/com/ello/webhook/*
Runtime	docker-compose.yml ve frontend/nginx.conf
Tests	backend/test, backend/src/**/*spec.ts, frontend/e2e/chat.spec.ts, java-webhook/src/test

Ek C. Komut Referansı

Amaç	Komut
Tüm geliştirme	`npm run dev`
Backend typecheck	`npm --prefix backend run typecheck`
Backend unit	`npm --prefix backend test`
Backend e2e	`npm --prefix backend run test:e2e`
Frontend typecheck	`npm --prefix frontend run typecheck`
Frontend unit	`npm --prefix frontend run test:ci`
Frontend Playwright	`npm --prefix frontend run test:e2e`
Java test	`cd java-webhook; ./mvnw.cmd test`
Prisma doğrulama	`npm --prefix backend run prisma:validate`
Database audit	`npm --prefix backend run db:audit`
Full stack	`npm.cmd run test:full-stack`
Compose durum	`docker compose ps`
Backend logs	`docker compose logs --tail 100 backend`
Java logs	`docker compose logs --tail 100 java-webhook`

Ek D. Sonuç

ello, kısa geliştirme süresine rağmen yalnızca statik bir frontend demosu değildir. Kalıcı ilişkisel veri modeli, role dayalı grup yönetimi, gerçek zamanlı mesaj güvenilirliği, dış sistem adaptörü, çoklu admin ticket koordinasyonu ve WebRTC çağrı kurtarma davranışıyla uçtan uca çalışan bir sistemdir.

En güçlü tarafı, özelliklerin ayrı ayrı görünmesinden çok aynı domain kuralları üzerinde birleşmesidir: frontend işlemi başlatır, NestJS yetki ve iş kuralını uygular, PostgreSQL state'i korur, Socket.IO diğer istemcileri günceller ve test paketi bu zinciri gerçek Docker ortamında doğrular.

Genel değerlendirme

Mevcut sürüm demo ve teknik sunum için yeterince kapsamlı ve doğrulanmıştır. Production'a geçişte ana ihtiyaç yeni sohbet özelliğinden çok oturum güvenliği, TURN/TLS, distributed realtime state ve object storage gibi operasyonel konulardır.